



Python 數據分析1: Xlwings & Openpyxl

By Wendy Chin
On Dec.,2nd,2022

时间元组表达

时间元组(9组数字)来处理时间， 例如t = (2016, 2, 17, 17, 3, 38, 1, 48, 0)

序号	字段	值	属性
0	4位数年	2008	tm_year
1	月	1 到 12	tm_mon
2	日	1到31	tm_mday
3	小时	0到23	tm_hour
4	分钟	0到59	tm_min
5	秒	0到61 (60或61 是闰秒)	tm_sec
6	一周的第几日	0到6 (0是周一)	tm_wday
7	一年的第几日	1到366 (儒略历)	tm_yday
8	夏令时	是否为夏令时， 值有： 1(夏令时)、0(不是夏令时)、 -1(未知)， 默认 -1	tm_isdst

datetime模組

- **datetime.date** :
表示日期的類，常用的屬性有year, month, day
- **datetime.datetime** :
表示日期時間的類，常用的屬性有hour, minute, second, microsecond
- **datetime.time** :
表示時間的類
- **datetime.timedelta** :
表示時間間隔，即兩個時間點的間隔。在日期上做天days，小時hour，分鐘，秒，毫秒，微妙的時間計算
- **datetime.tzinfo** :
時區的相關資訊

格式化时间： 表达代码

格式化日期時間輸入

格式代碼	內容說明	格式代碼	內容說明
%y	两位数的年份表示 (00-99)	%b	本地简化的月份名称
%Y	四位数的年份表示 (000-9999)	%B	本地完整的月份名称
%m	月份 (01-12)	%c	本地相应的日期表示和时间表示
%d	月内中的一天 (0-31)	%j	年内的一天 (001-366)
%H	24小时制小时数 (0-23)	%p	本地A.M.或P.M.的等价符
%I	12小时制小时数 (01-12)	%U	一年中的星期数 (00-53) 星期天为星期的开始
%M	分钟数 (00=59)	%w	星期 (0-6) , 星期天为星期的开始
%S	秒 (00-59)	%W	一年中的星期数 (00-53) 星期一为星期的开始
%a	本地简化星期名称	%x	本地相应的日期表示
%A	本地完整星期名称	%X	本地相应的时间表示
%%	%号本身	%Z	当前时区的名称

datetime.date

序号	函数	内容说明	範例
1	datetime.date(year, month, day)	傳入參數，返回一個新構造的日期	<pre>import datetime date1 = datetime.date(2022,11,25) print(date1) >>>2022-11-25</pre>
2	datetime.date.today()	返回一个表示当前本地日期的date对象	<pre>date1 = datetime.date.today() print(date1) >>>2022-11-25</pre>
3	datetime.date.fromtimestamp(timestamp)	根据给定的时间戳(秒數)，返回一个date对象。	
4	datetime.date.fromordinal(n)	傳入整數參數(時間間隔)，返回從公元1年1月1日起計算的時間	<pre>import datetime date1 = datetime.date.fromordinal(600) print(date1) >>>0002-08-23</pre>
5	datetime.date.timetuple()	返回日期对应的time.struct_time对象即一个元组	
6	datetime.date.toordinal():	與datetime.date.fromordinal(n)相反，返回n	
7	datetime.date.weekday()	返回weekday，如果是星期一，返回0；如果是星期二2，返回1，以此类推；	<pre>date1 = datetime.date.today() dateT = datetime.date.weekday(date1) print(dateT) >>>4</pre>
8	datetime.date.isoweekday()	返回weekday，如果是星期一，返回1；如果是星期二2，返回2，以此类推；	<pre>tm1 = datetime.date.today().isoweekday() print(tm1) >>>5</pre>
9	datetime.date.isocalendar()	返回格式如(year, month, day)的元组；	<pre>tm1 = datetime.date.today().isocalendar() print(tm1) >>>datetime.IsoCalendarDate(year=2022, week=47, weekday=5)</pre>
10	datetime.date.isoformat()	返回格式如'YYYY-MM-DD' 的字符串；	
11	datetime.date.ctime(date)	返回參數日期的起始時間	<pre>date1 = datetime.date.today() dateT = datetime.date.ctime(date1) print(dateT) >>>Fri Nov 25 00:00:00 2022</pre>
12	datetime.date.strftime(date,fmt)	自定义格式化字符串	<pre>date1 = datetime.date.today() dateT = datetime.date.strftime(date1,'%Y/%m/%d') print(dateT) >>>2022/11/25</pre>
13	datetime.date.replace(year, month, day):	生成一个新的日期对象，用参数指定的年，月，日代替原有对象中的属性。（原有对象仍保持不变	
14	datetime.date.year、 datetime.date.month、 datetime.date.day:	返回屬性(屬性無參數)：年、月、日	<pre>dateT = datetime.date.today().year print(dateT) >>>2022</pre>

datetime.time

序号	函数	内容说明	範例
1	<code>datetime.time.min</code>	返回属性: <code>time</code> 类所能表示的最小时间。	<pre>import datetime date1 = datetime.time.min print(date1); >>> 00:00:00</pre>
2	<code>datetime.time.max</code>	返回属性: <code>time</code> 类所能表示的最大时间。	<pre>dateT = datetime.time.max print(dateT) >>> 23:59:59.999999</pre>
3	<code>datetime.time.resolution</code>	返回属性: <code>time</code> 类表示时间的最小单位, 这里是1微秒	<pre>dateT = datetime.time.resolution print(dateT) >>> 0:00:00.000001</pre>
4	<code>datetime.time.hour</code> 、 <code>datetime.time.minute</code> 、 <code>datetime.time.second</code> 、 <code>datetime.time.microsecond</code>	返回属性: 时、分、秒、微秒;	<pre>tm1 = datetime.time(10,48,56) print(tm1.hour) >>> 10</pre>
5	<code>datetime.time.tzinfo</code>	时区信息	
6	<code>datetime.time.replace(hour=,minute=,second=,microsecond=)</code>	创建一个新的时间对象, 用参数指定的时、分、秒、微秒代替原有对象中的属性 (原有对象仍保持不变)	<pre>tm2 = datetime.time.replace(tm1,hour=12,minute=10,second=46,microsecond=38) print(tm2) >>> 12:10:46.000038</pre>
7	<code>datetime.time.isoformat()</code>	返回型如"HH:MM:SS"格式的字符串表示;	<pre>tm1 = datetime.time(10,48,56) tm2 = datetime.time.isoformat(tm1) print(tm2) >>> 10:48:56</pre>
8	<code>datetime.time.strftime(fmt)</code>	返回自定义格式化字符串	

datetime.datetime

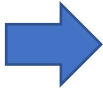
序号	函数	内容说明	範例
1	<code>datetime.datetime.min</code>	返回属性：所能表示的最小时间。	<pre>import datetime date1=datetime.datetime.min print(date1); >>>00:00:00</pre>
2	<code>datetime.datetime.max</code>	返回属性：所能表示的最大时间。	<pre>dateT=datetime.datetime.max print(dateT) >>>23:59:59.999999</pre>
3	<code>datetime.datetime.resolution</code>	返回属性：表示时间的最小单位，这里是1微秒	<pre>dateT=datetime.datetime.resolution print(dateT) >>>0:00:00.000001</pre>
4	<code>datetime.datetime.today()</code>	返回一个表示当前本地时间的datetime对象；	<pre>tm1=datetime.datetime.today() print(tm1) >>>2022-11-25 18:59:12.425442</pre>
5	<code>datetime.datetime.now([tz])</code>	返回一个表示当前本地时间的datetime对象，如果提供了参数tz，则获取tz参数所指时区的本地时间；	<pre>tm1=datetime.datetime.now() print(tm1) >>>2022-11-25 18:59:12.425442</pre>
6	<code>datetime.datetime.utcnow()</code>	返回一个当前utc时间的datetime对象；	
7	<code>datetime.datetime.fromtimestamp(timestamp[,tz])</code>	根据时间戳创建一个datetime对象，参数tz指定时区信息；	
8	<code>datetime.datetime.utcfromtimestamp(timestamp)</code>	根据时间戳创建一个datetime对象；	
9	<code>datetime.datetime.combine(date,time)</code>	根据date和time，创建一个datetime对象；	<pre>dt1=datetime.date(2022,12,12) tm1=datetime.time(11,12,28) tm2=datetime.datetime.combine(dt1,tm1) print(tm2) >>>2022-12-12 11:12:28</pre>
10	<code>datetime.datetime.strptime(date_string,format)</code>	将格式字符串转换为datetime对象(注意，字符串格式与参数中的格式要一致)	<pre>tm1='2022/11/25' tm2=datetime.datetime.strptime(tm1,'%Y/%m/%d') print(tm2) >>>2022-11-25 00:00:00</pre>
11	<code>datetime.datetime.year, month, day, hour, minute, second, microsecond, tzinfo</code>	返回属性：年、月、日、时、分、秒、毫秒、时区	<pre>tm2=datetime.datetime.today() print(tm2.year) >>>2022</pre>

12	<code>datetime.datetime.date()</code>	获取date对象；	<pre>tm1=datetime.datetime.today() tm2=datetime.datetime.date(tm1) print(tm2) >>>2022-11-25</pre>
13	<code>datetime.datetime.time()</code>	获取time对象；	<pre>tm1=datetime.datetime.today() tm2=datetime.datetime.time(tm1) print(tm2) >>>19:34:26.503045</pre>
14	<code>datetime.datetime.replace((hour=,minute=,second=,microsecond=))</code>	生成一个新的日期时间对象，用参数指定的：年、月、日、时、分、秒、毫秒、时区代替原有对象中的属性	
15	<code>datetime.datetime.timetuple()</code>	返回日期对应的datetime.struct_time对象即一个元组；	<pre>tm1=datetime.datetime.today().timetuple() print(tm1) >>>time.struct_time(tm_year=2022, tm_mon=11, tm_mday=25, tm_hour=19, tm_min=42, tm_sec=18, tm_wday=4, tm_yday=329, tm_isdst=-1)</pre>
16	<code>datetime.datetime.utctimetuple()</code>	返回一个当前utc时间的datetime.struct_time对象即一个元组	
17	<code>datetime.datetime.toordinal()</code>	与datetime.datetime.fromordinal(n)相反，返回n	<pre>tm1=datetime.datetime.today().toordinal() print(tm1) >>>738484</pre>
18	<code>datetime.datetime.weekday()</code>	返回weekday，如果是星期一，返回0；如果是星期二，返回1，以此类推	<pre>tm1=datetime.datetime.today().weekday() print(tm1) print(tm1) >>>4</pre>
19	<code>datetime.datetime.isocalendar()</code>	返回格式如(year, month, day, hour、minute、second、microsecond、tzinfo)的元组	<pre>tm1=datetime.datetime.today().isocalendar() print(tm1) >>>datetime.IsoCalendarDate(year=2022, week=47, weekday=5)</pre>
20	<code>datetime.datetime.isoformat()</code>		<pre>tm1=datetime.datetime.today().isoformat() print(tm1) >>>2022-11-25T19:37:54.172368</pre>
21	<code>datetime.datetime.ctime()</code>	返回一个日期时间的C格式字符串，等效于time.ctime(time.mktime(dt.timetuple()))	<pre>tm1=datetime.datetime.today().ctime() print(tm1) >>>Fri Nov 25 19:43:39 2022</pre>
22	<code>datetime.datetime.strftime(format)</code>	返回自定义格式化字符串	<pre>tm1=datetime.datetime.today().strftime('%Y/%m/%d') print(tm1) >>>2022/11/25</pre>

datetime.timedelta

序号	函数	内容说明	範例
1	<code>datetime.timedelta(days, seconds, microseconds, milliseconds, minutes, hours, weeks)</code>	表示时间差, 目前最大單位為weeks	<pre>import datetime #計算上個月最後一天日期 tm1=datetime.date.today() fsdate=datetime.date(tm1.year,tm1.month,1) lsdate=fsdate-datetime.timedelta(days=1) print(lsdate)</pre> >>>2022-10-31

```
#計算當月每周四日期
tday=datetime.date.today()
fsdate=datetime.date(tday.year,tday.month, day: 1)
fswkday=datetime.date.isoweekday(fsdate)
# sundate=fsdate-datetime.timedelta(days=fswkday)
# thdate=sundate+datetime.timedelta(days=4)
thdate=fsdate-datetime.timedelta(days=fswkday-4)
for i in range(0,6):
    everythdate=thdate+datetime.timedelta(i*7)
    if everythdate.month==tday.month:
        print(everythdate)
    elif everythdate.month>tday.month:
        break
```



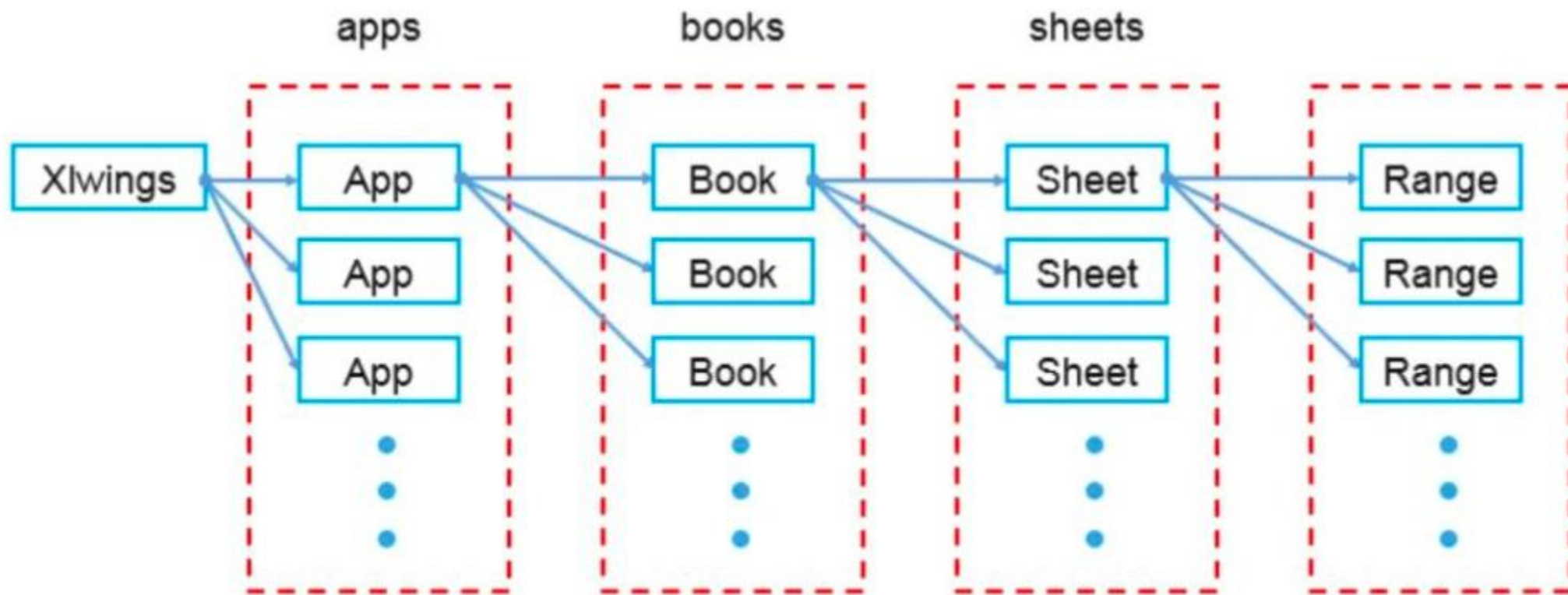
2025-07-03
2025-07-10
2025-07-17
2025-07-24
2025-07-31



Xlwings-簡介

xlwings：需要安裝有 Excel 軟件，支持 **.xls**和 **.xlsx** 格式；可以調用 Excel 文件中**VBA**寫好的程序；和 **matplotlib** 以及 **pandas** 的兼容性强

openpyxl：不需要 Excel 軟件，僅支持 **.xlsx** 格式



Xlwings- 引用

Title

import xlwings as xw ⊖

导入xlwings模块, 并简称为xw;
简称为xw, 是为了方便后续使用;

import os ⊖ 导入os模块

app = xw.App(visible = False, add_book = False) ⊖

visible: Ture可见, False不可见

add_book: Ture增加, False不增加

https://blog.csdn.net/m0_58112612

```
import xlwings as xw
import os
import warnings

path1="D:\\RPA Demo\\Excel\\"
fileNm1="VendorStock2021-Up.xlsx"
path2='C:/Users/wendy_chin/Desktop/Demo/Temp/'

excel=xw.App(visible=True, add_book=False)
```

Xlwings- 工作簿

除以下常用函数或方法外，excel工作簿还有：

- 激活时为类似 `wb.activate()` 或者类似 `wb=xw.books.active`
- 调用已经写好的VBA, 类似 `wb.macro(巨集名称)`

`excel_path = r'E:\Andy\Data Source\xlwings\Make Excel Fly\Test.xlsx'`

Excel工作簿的位置路径

打开

1 `wb = app.books.open(excel_path)`

wb: workbook的简称

2 `wb = xw.Book(excel_path)`

不推荐这种打开方式

新建

`wb_new = app.books.add()`

保存

`wb.save()` 原路径保存, 括号 () 中无需填写

`wb_new.save(r'E:\Andy\Data Source\xlwings\Make Excel Fly\wb_new.xlsx')`

新路径保存, 括号 () 中需要填写新的位置路径

关闭

`wb.close()`

退出

`app.quit()`

返回工作簿的名称

`wb.name`

名称带xlsx后缀

返回工作簿的绝对路径

`wb.fullname`

强制关闭

```
for wb in xw.apps.keys():  
    wb = 'taskkill/pid ' + str(wb) + ' -t -f'  
    os.system(wb)
```

Excel工作簿

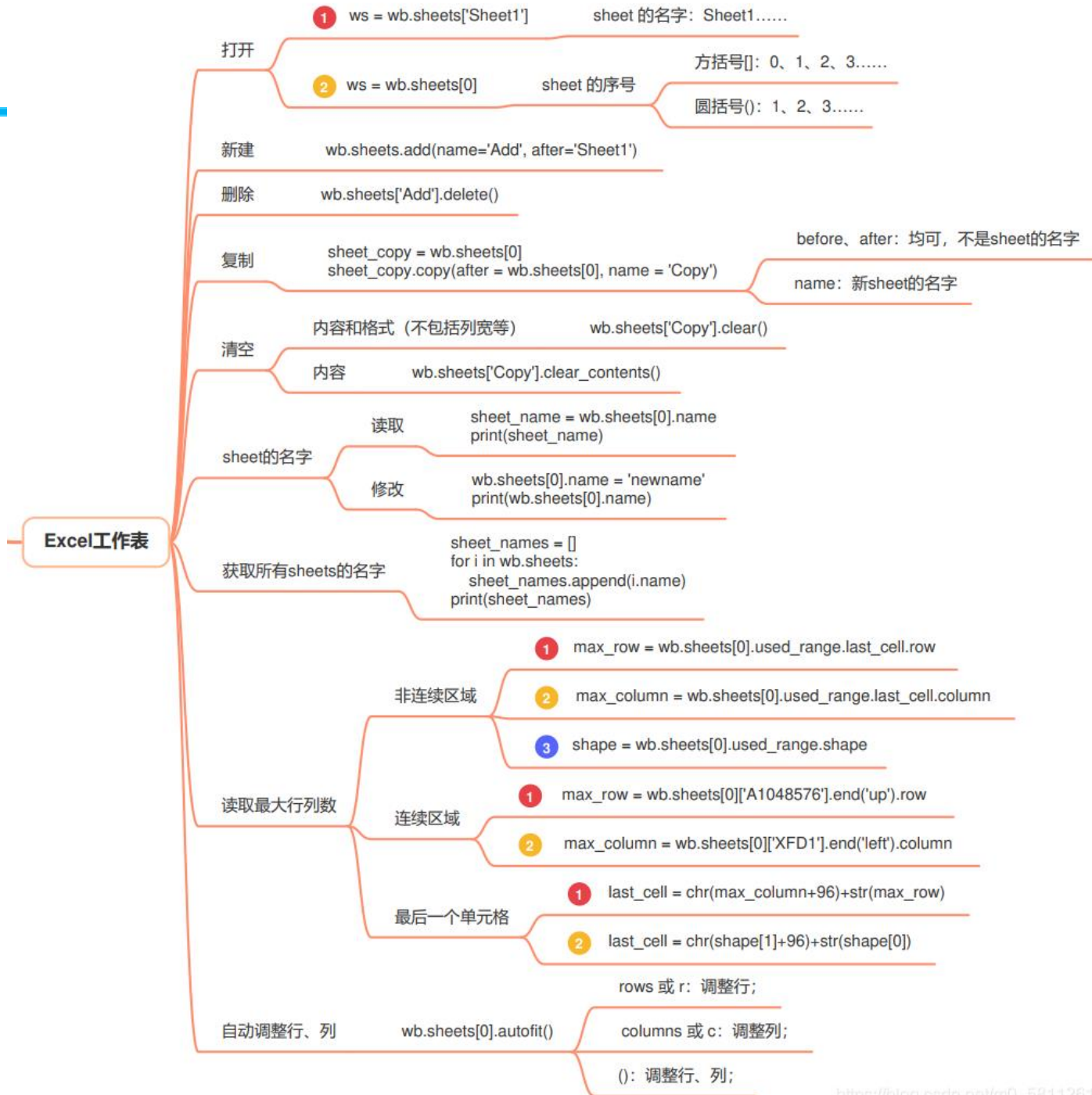
```
if not os.path.exists(path1+'~$'+fileNm1):  
    wkbook1=excel.books.open(path1+fileNm1)  
else:  
    wkbook1=xw.books[fileNm1]
```

Xlwings- 工作表

除左边常用函数或方法外，excel工作表激活時為類似 `ws.activate()` 或者類似 `ws=xw.books.sheets.active`; 计算工作表个数属性为 `.count`

```
for i in range(wkbook1.sheets.count):
    shtNm=wkbook1.sheets[i].name
    if 'Stock' in shtNm:
        worksheeti=wkbook1.sheets[i]
        worksheeti.range('7:7').delete()
        worksheeti.range('1:5').delete()
        worksheeti.range('C:D').delete()
        worksheeti.range('A:A').delete()
        lastrow=worksheeti.range('A1048576').end('up').row
        lastcol = worksheeti.range('XFD1').end('left').column
```

```
wkbook2=excel.books.add()
worksheet2=wkbook2.sheets.add(name='detail',after=wkbook2.sheets.count)
worksheet2['A1'].value=head1
worksheet2['A2'].value=data1
worksheet2.autofit()
```



Xlwt- 單元格属性、方法

Range属性或方法, 設rng=xw.sheets[0].range('A1')

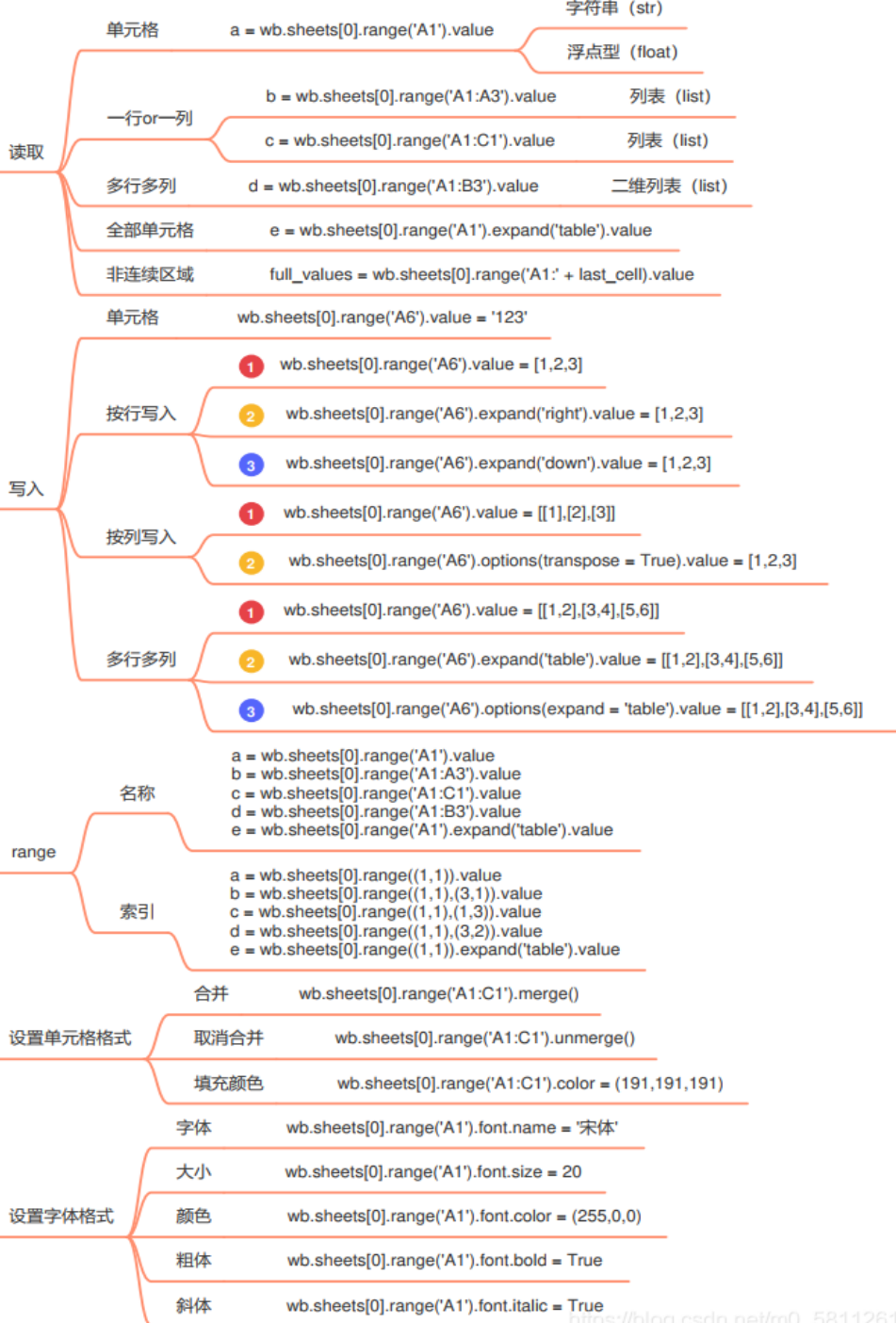
序号	名称	说明	範例
1	rng.add_hyperlink()	加入超链接	rng.add_hyperlink(r'www.baidu.com','百度', '提示: 点击即链接到百度')
2	rng.address rng.get_address()	取得当前range的地址	
3	rng.clear_contents()	清除range的内容	
4	rng.clear()	清除格式和内容	
5	rng.delete()	删除单元格	
6	rng.color	取得range的背景色,以元组形式返回RGB值, 如(255,0,255)	
7	rng.color=(RGB值)	设置range的颜色	rng.color=(255, 255, 255)
8	rng.color=None	清除range的背景色	
9	rng.column	获得range的第一列列标	
10	rng.count	返回range中单元格的个数	
11	rng.current_region	返回当前range所在区域的区域表达。如果是連續性數據, 則返回所連續的整片區域	
12	rng.end()	返回ctrl + 方向, 有參數down,up, left,right	
13	rng.formula	获取公式或者输入公式	rng.formula='=SUM(B1:B5)'
14	rng.formula_array	数组公式	
15	rng.get_address(row_absolute=True, column_absolute=True, include_sheetname=False, external=False)	获得单元格的绝对地址	
16	rng.column_width	获得列宽	
17	rng.width	返回range的总宽度	
18	rng.last_cell	获得range中右下角最后一个单元格	
19	rng.offset(a,b)	range平移,获取到当前range向右a格, 向下移动b格同样大小的那片区域, a,b可以为负值	
20	rng.resize(row_size=None, column_size=None)	进行resize改变range的大小	
21	rng.row	range的第一行行标	
22	rng.row_height	行的高度, 所有行一样高返回行高, 不一样返回None	
23	rng.height	返回range的总高度	
24	rng.shape	返回range的行数和列数	
25	rng.sheet	返回range所在的sheet	
26	rng.rows	返回range的所有行	
27	rng.rows.count	返回range的列数	
28	rng.autofit()	所有行、列宽度自适应	
29	rng.columns.autofit()	所有列宽度自适应	
30	rng.rows.autofit()	所有行宽度自适应	
31	rng.number_format	獲取數值格式	
32	rng.insert()	插入行、列	

單元格常用方法与属性可参考左右两个表格.

Python异常错误处理:
try:
except:
finally:

```
wksheeti.range(1,lastcol+1).value='Aging'  
grp=wksheeti.range('D2:D'+str(lastrow)).value  
group=sorted(set(grp)) # set去掉重复字符串,并按照默认字母升序排序  
for x in group:  
    wkbookx=excel.books.add()  
    wksheetx=wksheeti.sheets[0]  
    wksheetx.range(1,1).value=wksheeti.range((1,1),(1,lastcol+1)).value  
    a=1  
    for j in range(2,lastrow+1):  
        try:  
            wksheeti.range(j,lastcol+1).formula='=today()-'+chr(lastcol+96)+str(j)  
            ...  
        except:  
            pass  
  
    if wksheeti.range(j,4).value==x:  
        a+=1  
        wksheetx.range(a,1).value=wksheeti.range((j,1),(j,lastcol+1)).value  
  
wkbookx.save(path2+x+'.xlsx')  
wkbookx.close()
```

Excel单元格



Xlwings- 格式调整

格式调整主要用api

"""设置边框"""

Borders(9) 底部边框, LineStyle = 1 直线。

b3.api.Borders(9).LineStyle = 1

b3.api.Borders(9).Weight = 3 # 设置边框粗细。

Borders(7) 左边框, LineStyle = 2 虚线。

b3.api.Borders(7).LineStyle = 2

b3.api.Borders(7).Weight = 3

Borders(8) 顶部框, LineStyle = 5 双点划线。

b3.api.Borders(8).LineStyle = 5

b3.api.Borders(8).Weight = 3

Borders(10) 右边框, LineStyle = 4 点划线。

b3.api.Borders(10).LineStyle = 4

b3.api.Borders(10).Weight = 3

Borders(5) 单元格内从左上角 到 右下角。

b3.api.Borders(5).LineStyle = 1

b3.api.Borders(5).Weight = 3

'''跨表格复制、粘贴'''

my_values = sht_1.range('a2: d4').options(ndim=2).value

sht_2.range('a1').value = my_values

sht.range(3,2).value = 'b3'

"""设置单元格大小"""

sht.autofit() # 自动调整单元格大小。注：此方法是在单元格写入内容后，再使用，才有效。

sht.range(1,4).column_width = 5 # 设置第4列 列宽。 (1,4) 为第1行第4列的单元格

sht.range(1,4).row_height = 20 # 设置第1行 行高

"""设置单元格 字体格式"""

b3.color = 255,200,255 # 设置单元格的填充颜色

b3.api.Font.ColorIndex = 3 # 设置字体的颜色，具体颜色索引见下方。

b3.api.Font.Size = 24 # 设置字体的大小。

b3.api.Font.Bold = True # 设置为粗体。

b3.api.HorizontalAlignment = -4108 # -4108 水平居中。 -4131 靠左， -4152 靠右。

b3.api.VerticalAlignment = -4130 # -4108 垂直居中（默认）。 -4160 靠上， -4107 靠下。

b3.api.NumberFormat = "0.00" # 设置单元格的数字格式。

"""合并拆分单元格"""

sht.range('C8:D8').api.merge() # 合并单元格 C8 到 D8

sht.range('C8:D8').api.unmerge() # 拆分单元格。

'''插入 、删除 一行'''

sht1.range('a3').api.EntireRow.Delete() # 会删除 'a3' 单元格所在的行。

sht1.api.Rows(3).Insert() # 会在第3行插入一行，原来的第3行下移。

'''插入 、删除 一列'''

sht1.range('c2').api.EntireColumn.Delete() # 会删除 'c2' 单元格所在的列。

sht1.api.Columns(3).Insert() # 会在第3列插入一列，原来的第3列右移。

Xlwings- 格式调整

```
'''排序，删除重复值'''
# 排序使用方法：
# 1、选择需要排序的区域。这里用 'a2' 是因为排序的数据是从第二行开始的，第一行是标题，不应该
# 2、选择按那一列进行排序 Key1=sht.range('c2').api, 这里选择的是按 第 c 列排序，所以这里
# 3、Order1=1 为升序，2为降序。
sht1.range('a2',(rows,columns)).api.Sort(Key1=sht.range('c2').api, Order1=1)

# 删除重复值使用方法：
# RemoveDuplicates(3) 为按第3列内容进行删除重复项。
sht1.range('a2',(rows,columns)).api.RemoveDuplicates(3)

"""插入、读取公式"""
sht1.range('d1').formula = '=sum(e1+f1)'    # 插入公式
print(sht1.range('d1').formula)

'''同个表格复制、粘贴'''
# 复制 a2 到 a6 之间单元格的值，粘贴到'a15'中
sht.range('a2','a6').api.Copy(sht.range('a15').api)
```

Xlwings- 產生圖表並另存圖片

chart_type list:

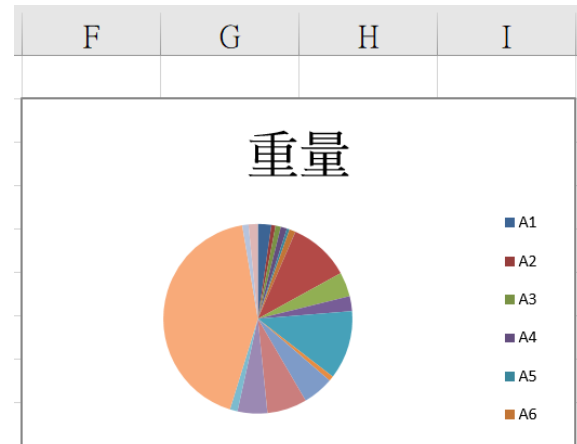
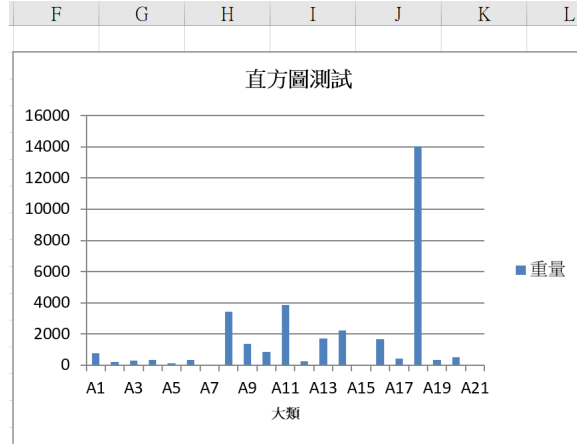
```
excelapp=xw.App(visible=True,add_book=False)
wb1=excelapp.books.open(path+f'lnm3')
sht1=wb1.sheets[0]
data1=sht1.range("A1:B22")

#新增圖表
chart1=sht1.charts.add(left=sht1.range("F2").left,
                        top=sht1.range("F2").top,
                        width=320,height=220)
chart2=sht1.charts.add(left=sht1.range("F18").left,
                        top=sht1.range("F18").top,
                        width=sht1.range("F18:I25").width,
                        height=sht1.range("F18:I25").height)

#設置數據來源
chart1.set_source_data(data1)
chart2.set_source_data(data1)

#設置圖表類型
chart1.chart_type='column_clustered'
chart2.chart_type='pie'

#調用api設置圖表標題與坐標軸標題
# api[1]表示調用excel API接口, Axes(1)表示X軸, Axes(2)為Y軸
chart1.api[1].SetElement(2)
chart1.api[1].ChartTitle.Text="直方圖測試"
chart1.api[1].ChartTitle.Font.Size=12
chart1.api[1].Axes(1).HasTitle=True
chart1.api[1].Axes(1).AxisTitle.Text="大類"
chart1.api[1].Axes(1).AxisTitle.Font.Size=8
# chart1.api[1].Axes(1).Format.TextFrame2.TextRange.Font.Size = 6
chart2.api[1].Legend.Font.Size=6 #圖例設置字體大小
#圖表另存為圖片
chart1.to_png(path+"chart1.png")
```



```
'xy_scatter': -4169,
'xy_scatter_lines': 74,
'xy_scatter_lines_no_markers': 75,
'xy_scatter_smooth': 72,
'xy_scatter_smooth_no_markers': 73
```

```
'3d_area': -4098,
'3d_area_stacked': 78,
'3d_area_stacked_100': 79,
'3d_bar_clustered': 60,
'3d_bar_stacked': 61,
'3d_bar_stacked_100': 62,
'3d_column': -4100,
'3d_column_clustered': 54,
'3d_column_stacked': 55,
'3d_column_stacked_100': 56,
'3d_line': -4101,
'3d_pie': -4102,
'3d_pie_exploded': 70,
'area': 1,
'area_stacked': 76,
'area_stacked_100': 77,
'bar_clustered': 57,
'bar_of_pie': 71,
'bar_stacked': 58,
'bar_stacked_100': 59,
'bubble': 15,
'bubble_3d_effect': 87,
'column_clustered': 51,
'column_stacked': 52,
'column_stacked_100': 53,
'cone_bar_clustered': 102,
'cone_bar_stacked': 103,
'cone_bar_stacked_100': 104,
'cone_col': 105,
```

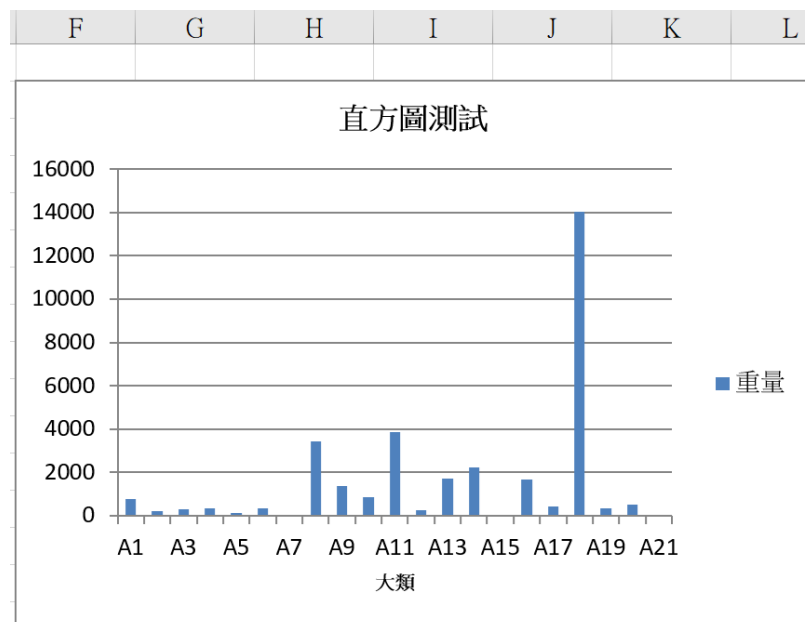
```
'cone_col_clustered': 99,
'cone_col_stacked': 100,
'cone_col_stacked_100': 101,
'cylinder_bar_clustered': 95,
'cylinder_bar_stacked': 96,
'cylinder_bar_stacked_100': 97,
'cylinder_col': 98,
'cylinder_col_clustered': 92,
'cylinder_col_stacked': 93,
'cylinder_col_stacked_100': 94,
'doughnut': -4120,
'doughnut_exploded': 80,
'line': 4,
'line_markers': 65,
'line_markers_stacked': 66,
'line_markers_stacked_100': 67,
'line_stacked': 63,
'line_stacked_100': 64,
'pie': 5,
'pie_exploded': 69,
'pie_of_pie': 68,
'pyramid_bar_clustered': 109,
'pyramid_bar_stacked': 110,
'pyramid_bar_stacked_100': 111,
'pyramid_col': 112,
'pyramid_col_clustered': 106,
'pyramid_col_stacked': 107,
'pyramid_col_stacked_100': 108,
'radar': -4151,
'radar_filled': 82,
'radar_markers': 81,
```

Xlwings- 插入圖片

- `books[x].sheets[y].pictures.add(图片来源, left, top, width, height, name)`

- ❑ 图片来源: 1.已经保存的图片文件, 需写完整的绝对路径; 2.数据流中产生的图, 如matplotlib已产生的plt/fig
- ❑ Left/top: 到excel中表的左边距/上边距的数值
- ❑ Width/height: 宽/高, 其数值为excel中的磅值
- ❑ Name: 对该图片命名, 可方便搜索

```
excelapp=xw.App(visible=True,add_book=False)
wb1=excelapp.books.open(path+f'nm3')
sht1=wb1.sheets[0]
...
sht1.pictures.add(path+"chart1.png",
                  left=sht1.range("F27").left,
                  top=sht1.range('F27').top,
                  width=320,height=220,name='testpic')
```



Openpyxl-簡介

openpyxl是一个用于读取/编写Excel 2010 xlsx/xlsm/xltx/xltn文件的Python库。它是由于缺乏从Python中读取/编写Office Open XML格式的现有库而诞生的。所有这些都归功于PHPExcel团队，因为openpyxl最初是基于PHPExcel的。以下是处理excel文件各库的优缺点：

早起Python	可处理的Excel文件后缀		读取		写入	修改	保存	样式调整	插入图片
	.xls	.xlsx	读取	消耗时间 以10MB .xlsx 文件为例					
xlrd	✓	✓	✓	12.38s	✗	✗	✗	✗	✗
xlwt	✓	✗	✗	-	✓	✓	✓	✓	✓
xlutils	✓	✗	✗	-	✓	✓	✓	✗	✗
xlwings	✓	✓	✓	7.06s	✓	✓	✓	✓	✓
XlsxWriter	✗	✓	✗	-	✓	✗	✓	✓	✓
openpyxl	✗	✓	✓	27.93s	✓	✓	✓	✓	✓
pandas	✓	✓	✓	17.55s	✓	✗	✓	✗	✗

Openpyxl-工作簿、工作表

Openpyxl特点是数据调用都在后台。即，无法直接看到excel的编辑实时变化，想要确认是否bug, 用print()显示。因此，不存在关闭工作簿的命令代码。

```
import openpyxl as opx
from openpyxl.styles import Font, Color, Fill, Alignment
```

```
# 创建新的工作簿
wb=opx.Workbook()

#打开已有的xlsx文件（打开的文件加载在后台,无法直接看到视窗）
wb1=opx.load_workbook(filename=path1+fileNm1)
```

```
#获取当前活动表
ws=wb1.active
```

```
#新增工作表
ws1=wb1.create_sheet('openpytest1') #默认放最后一个
ws2=wb1.create_sheet('openpytest2', 0) # 放在第一个
ws3=wb1.create_sheet('openpytest3', -1) # 倒数第二个
```

```
#引用工作表
ws4=wb1.worksheets[0] #索引引用
ws5=wb1['VendorStock2021'] #表名引用
wslist=wb1.sheetnames #获取所有的表名称
ws6=wb.worksheets[0]
```

```
#修改、移动、复制、删除工作表
ws6.title='openpyts1' #修改表名
wb.move_sheet('openpyts1', -1) #向前移动指定表格，正值往后，负值往前
cpsheet=wb.copy_worksheet(ws6) #复制工作表
wb.remove('openpyts1') #删除指定表格名
del wb.worksheets[0] #删除指定索引的表格
```


Openpyxl-单元格、行、列之引用与属性

```
# 单元格、行、列的引用
cell1=ws5['B8']
cell2=ws5.cell(8,2) #用cell方法时,
row1=ws5[7]         #引用一行
row2=ws5[1:5]       #引用多行
col1=ws5['A']        #引用一列
col2=ws5['C:D']      #引用多列
rng1=ws5['A1:D8']    #引用一个区域范围
max_row=ws5.max_row  #最大行数
max_col=ws5.max_column #最大列数
```

单元格、行、列的引用
与相关属性，分别参考
上边、右边范例

```
#单元格、行、列属性
rowall=ws5.rows      #返回所有行，生成元组，若要显示时，需用tuple()
colall=ws5.columns   #返回所有列，生成元组，若要显示时，需用tuple()
cellvalues=ws5.values #返回所有的数据值，若要显示时，需用tuple()
cellmerge=ws5.merged_cells #返回所有的合并的单元格，若要显示时，需用tuple()
rowheight=ws5.row_dimensions[1] #返回第一行直径，生成元组，若要显示时，需用tuple()
colheight=ws5.column_dimensions['A'] #返回第一列直径，生成元组，若要显示时，需用tuple()
cell1.value='2'      #单元格赋值，当用cell()时，必须用.value
cell2.value=2        #单元格赋值，当用cell()时，必须用.value
colind=cell1.col_idx #单元格列索引
colind=cell1.column  #单元格列索引，返回2
rowind=cell1.row      #单元格行索引，返回8
colNm=cell1.column_letter #单元格列名，返回B
cellCo=cell1.coordinate #单元格坐标，返回B8
celltype=cell1.data_type #单元格数据类型，n:数值 s:字符串 d:日期时间
cellhasstyle=cell1.has_style #单元格是否有格式，默认False
cellstyle=cell1.style    #单元格格式，默认Normal
cellfont=cell1.font      #单元格字体属性
cellalign=cell1.alignment #单元格对齐属性
cellborder=cell1.border  #单元格边框属性
cellfill=cell1.fill      #单元格填充属性
```

Openpyxl-单元格、行、列之方法

```
#单元格、行、列方法
ws5.append([1,2,3])      #增加行
ws5.insert_cols(3)       #插入列
ws5.insert_rows(3)       #插入行
ws5.delete_cols(3)       #删除一列
ws5.delete_rows(7)       #删除一行
ws5.merge_cells('A1:B3') #合并单元格
ws5.move_range('B2:D4', rows=1, cols=2) #移动单元格，正值为行向下，列向右，负值相反
ws5.cell(1,1).font=Font(name='calibra', size=12, bold=True) #字体属性设置
rowheight.height=15      #设置行高，生成元组，若要显示时，需用tuple()
colheight.width=30       #设置列宽，生成元组，若要显示时，需用tuple()
```

Openpyxl-style 样式范例

```
fill = PatternFill(fill_type=None,
..             start_color='FFFFFFF',
..             end_color='FF000000')
border = Border(left=Side(border_style=None,
..             color='FF000000'),
..             right=Side(border_style=None,
..             color='FF000000'),
..             top=Side(border_style=None,
..             color='FF000000'),
..             bottom=Side(border_style=None,
..             color='FF000000'),
..             diagonal=Side(border_style=None,
..             color='FF000000'),
..             diagonal_direction=0,
..             outline=Side(border_style=None,
..             color='FF000000'),
..             vertical=Side(border_style=None,
..             color='FF000000'),
..             horizontal=Side(border_style=None,
..             color='FF000000'))
```

```
from openpyxl.styles import PatternFill, Border, Side, Alignment, Protection, Font
font = Font(name='Calibri',
...             size=11,
...             bold=False,
...             italic=False,
...             vertAlign=None,
...             underline='none',
...             strike=False,
...             color='FF000000')

alignment=Alignment(horizontal='general',
...             vertical='bottom',
...             text_rotation=0,
...             wrap_text=False,
...             shrink_to_fit=False,
...             indent=0)

number_format = 'General'
protection = Protection(locked=True,
...             hidden=False)
```

課後作業

1. Python代碼：羅列當月起第3個月每周二的日期
2. Python代碼：操作所發excel《shifttime.xlsx》:找出出勤異常的記錄並寫入表<exception>中。
3. Python代碼：操作所發excel《VendorStock-Up.xlsx》：
 - 3.1) 查找含有“Stock”的表名，
 - 3.2) 找到Item 3.1的工作表后，查找距当前日期呆滯天數超過180天的清單
 - 3.3) 找到Item 3.2的數據后，以“StorageBin”的地点代碼拆分成不同文件，
文件名稱以“StorageBin”下的地点命名.xlsx，例如“A1.xlsx”
4. 根据作业3，将拆分后的文件再合并成新文件“VS.xlsx”

Thank You